Lowest Common Ancestor of a Binary Search Tree (medium)

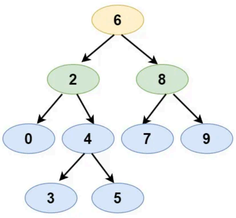
Problem Statement

Given a binary search tree (BST) and two of its nodes, find the node that is the lowest common ancestor (LCA) of the two given nodes. The LCA of two nodes is the node that lies in between the two nodes in terms of value and is the furthest from the root. In other words, it's the deepest node where the two nodes diverge in the tree. Remember, in a BST, nodes have unique values.

Examples

1.

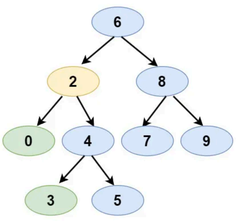
- Input:
  - BST: [6,2,8,0,4,7,9,null,null,3,5]
  - Node 1: 2
  - Node 2: 8
- Expected Output: 6



- Justification: The nodes 2 and 8 are on the left and right children of node 6. Hence, node 6 is their LCA.

2.

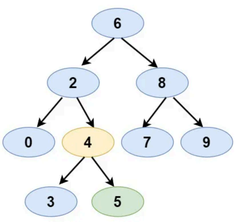
- Input:
  - BST: [6,2,8,0,4,7,9,null,null,3,5]
  - Node 1: 0
  - Node 2: 3
- Expected Output: 2



- Justification: The nodes 0 and 3 are on the left and right children of node 2, which is the closest ancestor to these nodes.

3.

- Input:
  - BST: [6,2,8,0,4,7,9,null,null,3,5]
  - Node 1: 4
  - Node 2: 5
- Expected Output: 4



- Justification: Node 5 is the right child of node 4. Hence, the LCA is node 4 itself.

Try it yourself

Try solving this question here:

```
Java
1  // class TreeNode {
2  //     int val;
3  //     TreeNode left;
4  //     TreeNode right;
5  //     TreeNode(int x) { val = x; }
6  // }
7
8  class Solution {
9      // Main function to determine LCA
10     public int lowestCommonAncestor(TreeNode root, int p, int q) {
11         // ToDo: Write Your Code Here.
12         return -1;
13     }
14 }
15
```

Test Case | Run Code Result | Stdout

```
[6,2,8,0,4,7,9,null,null,3,5]
2
8
```

Use Example Testcases | Run | Submit